

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Mohd. Razia Alangir Banu
Student Name	Y. Yashwanth
Roll Number	2520030506
Branch	CSE
Course Outcome	CO-4

Case Study Topic: Optimizing Multi-Attribute Inventory Sorting and Logistics Valuation via Radix Sort

Work Done Summary:

A high-throughput automated fulfillment center processes hundreds of thousands of retail packages per hour. Each package contains a structured 9-digit alphanumeric tracking identifier (the composite key). This key embeds critical logistical properties: the first 3 digits represent the destination zone code (000–999), the middle 3 digits represent the warehouse aisle position (000–999), and the final 3 digits signify the priority handling tier (000–999).

To stage packages correctly for automated robotic conveyor arms, the central logistics pipeline must sort these tracking keys in ascending order. The operations team faces three critical engineering constraints:

1. **Ultra-Low Latency:** High-speed conveyors leave a window of only fractions of a millisecond to sort batches containing up to 500,000 packages. Comparison-based sorting algorithms like Quick Sort or Merge Sort are causing CPU scheduling bottlenecks due to their $O(n \log n)$ performance boundaries.
2. **Strict Memory Footprint:** The processing nodes run on embedded industrial PCs with tightly budgeted memory buffers. Algorithms requiring massive dynamic allocations risk buffer overflows.
3. **Stability Requirement:** The conveyor sorting layout depends on maintaining the original relative order of packages that share identical destination zone codes (preserving their sequential timestamp arrival order). Thus, the underlying sorting engine must be strictly stable.

Implementation Details:

- Step-by-Step State Trace of LSD Radix Sort
- Initial Input Array: [329, 457, 657, 839, 436]
- Pass 1: Sorting by the units digit (Least Significant Digit)
 - Extract units digits: 329, 457, 657, 839, 436
 - Stable sort results: [436, 457, 657, 329, 839]
- (Note: 457 remains ahead of 657 and 329 remains ahead of 839 due to stability)
- Pass 2: Sorting by the tens digit (Middle Digit)
 - Extract tens digits from the current state: 436, 457, 657, 329, 839
 - Stable sort results: [329, 436, 839, 457, 657]

- Pass 3: Sorting by the hundreds digit (Most Significant Digit)
 - Extract hundreds digits from the current state: 329, 436, 839, 457, 657
 - Stable sort results: [329, 436, 457, 657, 839]
- The array is now fully sorted in ascending order across all three attributes.

```
import java.util.Arrays;
```

```
public class LogisticsRadixSorter {
```

```
/**
```

```
 * Core stable Counting Sort subroutine tailored to isolate and sort a single digit place.
```

```
 * @param array The target inventory array to sort.
```

```
 * @param exp The current exponent representing the digit place value (1, 10, 100, etc.)
```

```
 */
```

```
private static void stableCountingSortByDigit(int[] array, int exp) {
```

```
    int n = array.length;
```

```
    int[] output = new int[n];
```

```
    int[] count = new int[10]; // Base-10 digits (0-9)
```

```
    Arrays.fill(count, 0);
```

```
    // 1. Accumulate frequencies of the specific digit place
```

```
    for (int i = 0; i < n; i++) {
```

```
        int digit = (array[i] / exp) % 10;
```

```
        count[digit]++;
```

```
    }
```

```
    // 2. Transform counts to prefix sums to determine actual output placement indexes
```

```
    for (int i = 1; i < 10; i++) {
```

```
        count[i] += count[i - 1];
```

```
    }
```

```
    // 3. Build the output array in REVERSE order to preserve stability
```

```
    for (int i = n - 1; i >= 0; i--) {
```

```
        int digit = (array[i] / exp) % 10;
```

```
        output[count[digit] - 1] = array[i];
```

```
        count[digit]--;
```

```
    }
```

```
    // 4. Transfer sorted state back into original array buffer
```

```
    System.arraycopy(output, 0, array, 0, n);
```

```
}
```

```
/**
```

```
 * Executes an LSD Radix Sort across the array entries.
```

```
 */
```

```
public static void radixSort(int[] array) {
```

```
    if (array == null || array.length == 0) return;
```

```
    // Find the maximum value to determine how many digit positions exist
```

```
    int maxVal = array[0];
```

```
    for (int val : array) {
```

```
        if (val > maxVal) {
```

```
            maxVal = val;
```

```
        }
```

```
    }
```

```

int passCounter = 1;
// Loop through each digit place value (1s, 10s, 100s...)
for (int exp = 1; maxVal / exp > 0; exp *= 10) {
    stableCountingSortByDigit(array, exp);
    System.out.println("State after Pass " + passCounter + " (Digit Exp " + exp + "): " + Arrays.toString(array));
    passCounter++;
}
}

public static void main(String[] args) {
    int[] inventoryBatch = {329, 457, 657, 839, 436};

    System.out.println("Initial Package Inventory Array: " + Arrays.toString(inventoryBatch));
    System.out.println("-----");

    radixSort(inventoryBatch);

    System.out.println("-----");
    System.out.println("Final Validated Sorted Package Array: " + Arrays.toString(inventoryBatch));
}
}

```

Result: Screenshots / Output

- Plaintext
- Initial Package Inventory Array: [329, 457, 657, 839, 436]
- -----
- State after Pass 1 (Digit Exp 1): [436, 457, 657, 329, 839]
- State after Pass 2 (Digit Exp 10): [329, 436, 839, 457, 657]
- State after Pass 3 (Digit Exp 100): [329, 436, 457, 657, 839]
- -----
- Final Validated Sorted Package Array: [329, 436, 457, 657, 839]

FS

GitHub Link:

Student Signature	Faculty Signature
-------------------	-------------------